

ARCHITECTURE REPORT

Global ERP — Scalability & Performance Architecture Plan

 March 24, 2026  PostgreSQL · Next.js · Multi-tenant SaaS  164 migrations · Shared-schema design

Current Architecture Overview

Global ERP uses a **shared-schema multi-tenant PostgreSQL** database — all companies share the same tables, isolated by `company_id`. This is the correct industry-standard approach used by Shopify, GitHub, and Linear. The goal is *not* to separate into per-company databases, but to fix real bottlenecks and add the right infrastructure layers to scale to hundreds of companies.

Key insight: The data model is sound. The problems are in the *infrastructure around it* — connection pooling, caching, and query routing.

Current Weaknesses Identified

CRITICAL

DB Connection Pool = 1
`packages/ai-core/src/db.ts`

CRITICAL

No Caching Layer (Redis)
Entire application

MEDIUM

No Read/Write Separation
All list & report queries

MEDIUM

No Table Partitioning
leads, customers, calls, inspections

MEDIUM

No Background Job Queue
Campaign sends, bulk reports

Why NOT Separate Databases Per Company?

Approach	Pros	Cons
Shared schema (current) ✓	Simple · 1 migration to deploy · easy cross-company reporting	Needs indexing + partitioning

Approach	Pros	Cons
Schema-per-tenant	Better isolation	Complex migrations · harder to query across companies
DB-per-tenant	Total isolation	100 companies = 100 connection pools · migration nightmare · 10× infra cost

Decision: Stay with shared schema — fix the infrastructure around it.

Recommended Implementation Tiers

TIER 1

Critical Fixes — Immediate, High Impact

1

Fix Connection Pooling

PROBLEM

`max: 1` means every concurrent API request queues behind the previous one. Next.js serverless API routes may each spawn a new Postgres connection, quickly exhausting the database's connection limit.

SOLUTION

Use **PgBouncer** (transaction-mode) in front of PostgreSQL, or a managed pooler:

- **Supabase** — built-in PgBouncer, enable transaction pooling in dashboard
- **Neon** — built-in serverless pooler (`?pgbouncer=true` in connection string)
- **Self-hosted** — deploy PgBouncer sidecar with `pool_mode = transaction`

CODE CHANGE IN DB.TS

```
// Before – single connection, everything queues
const sql = postgres(connectionString, { max: 1 });

// After – proper pool for serverless Next.js
const sql = postgres(connectionString, {
  max: 10,           // pool size per serverless instance
  idle_timeout: 20,  // release idle connections after 20s
  connect_timeout: 10,
});
// Point connection string to PgBouncer port 6432, not Postgres 5432
```

2 Redis Caching Layer

PROBLEM

Every page load hits PostgreSQL, even for data that rarely changes (company config, permissions, chart of accounts).

WHAT TO CACHE (IMMEDIATE WINS)

Data	TTL
Company configuration & VIN credentials	5 min
User session + RBAC permissions	1 min
VIN lookup results	30 days
Branch list per company	5 min
Chart of Accounts per company	10 min

IMPLEMENTATION PATTERN

Add `ioredis` or `@upstash/redis` (serverless-friendly) to `packages/ai-core`. Create a `withCache()` wrapper:

```
// packages/ai-core/src/cache.ts
export async function getCompanyById(companyId: string) {
  return withCache(`company:${companyId}`, 300, () =>
    sql`SELECT * FROM companies WHERE id = ${companyId}`
  );
}
```

TIER 2 Scalability Improvements — 1 to 3 Months

3 PostgreSQL Table Partitioning by company_id

TARGET TABLES

Tables that grow proportionally with companies × activity: `leads`, `lead_events`, `customers`, `calls`, `inspections`, `work_orders`, `accounting_journal_lines`

STRATEGY — PARTITION BY LIST (COMPANY_ID)

```
-- Each company gets its own partition
CREATE TABLE leads_partitioned (LIKE leads)
  PARTITION BY LIST (company_id);

CREATE TABLE leads_company_1
  PARTITION OF leads_partitioned FOR VALUES IN ('uuid-1');

-- Default catches all new companies automatically
CREATE TABLE leads_default
  PARTITION OF leads_partitioned DEFAULT;
```

BENEFITS

- Queries with `WHERE company_id = $1` hit only that company's partition → smaller index, faster scan
- VACUUM and ANALYZE run per-partition → less lock contention
- Archive or delete a company by dropping their partition
- **Migration path:** Non-breaking — new migration + data copy + rename swap

4 Read Replica for Reports & Lists

PROBLEM

Heavy reads (customer lists with 10k+ rows, accounting reports, dashboard aggregates) compete with transactional writes (lead updates, invoice creation) on the same primary DB node.

SOLUTION

Add a PostgreSQL read replica and split connections in `db.ts`:

```
// packages/ai-core/src/db.ts
export const sql = postgres(process.env.DATABASE_URL!); // primary (writes)
export const readSql = postgres(process.env.DATABASE_READ_URL!); // replica (reads)
```

Route all `SELECT`-only queries in list/report repositories to `readSql`.

Managed options: Supabase read replicas · Neon branching · AWS RDS read replica · Railway

5

Background Job Queue for Heavy Operations

PROBLEM

Operations like campaign sends (10k+ push notifications), large report generation, and bulk CSV imports block API response threads and cause timeouts.

RECOMMENDED TOOLS

- **BullMQ** (Redis-backed) — best for Node.js, works cleanly in monorepo
- **Inngest** — serverless-friendly, no Redis required

MOVE THESE TO BACKGROUND JOBS

- Campaign send operations (marketing module)
- Large report generation (accounting, HR)
- Bulk customer imports
- VIN batch lookups

TIER 3

Enterprise Scale — Only if 500+ Companies

- **Schema-per-tenant migration** — each company gets `schema_<company_id>` PostgreSQL schema. Complex but enables per-company backups and true isolation.
- **TimescaleDB** — for `lead_events` , `calls` , `accounting_journal_lines` (time-series at scale).
- **Horizontal sharding via Citus** — distributes tables across multiple PG nodes by `company_id` .

Critical Files to Modify

File	Change
<code>packages/ai-core/src/db.ts</code>	Fix pool settings (<code>max: 10</code>), add <code>readSql</code> export
<code>packages/ai-core/src/index.ts</code>	Export cache utilities
<code>packages/ai-core/src/cache.ts</code> <i>(new)</i>	Redis <code>withCache()</code> wrapper
<code>migrations/165_partition_leads.sql</code> <i>(new)</i>	Partition leads table by <code>company_id</code>
<code>migrations/166_partition_customers.sql</code> <i>(new)</i>	Partition customers table by <code>company_id</code>

Verification & Testing Plan

1. **Connection pool:** Use `pg_stat_activity` to observe connection count. Run k6 or Artillery with 50 concurrent requests — response time should drop dramatically.
2. **Cache:** Add hit/miss logging to `withCache()`. Expect 50–80% cache hit rate on company config and permissions.
3. **Partitioning:** Run `EXPLAIN ANALYZE` on `SELECT * FROM leads WHERE company_id = $1` — should show partition pruning ("Seq Scan on leads_company_X" instead of full table scan).
4. **Read replica:** Monitor primary DB CPU during report generation — should drop significantly once reads route to replica.

Summary — Priority Order

- **Fix `max: 1` connection pool** — immediate win, zero business logic changes required. Single most impactful fix.
- **Add Redis caching** — cuts 50–80% of redundant DB reads. Company config, permissions, and VIN results are the biggest wins.
- **Table partitioning** — critical once any single company exceeds 100k rows in leads, customers, or journal lines.
- **Read replica** — add when dashboard and reports start impacting transactional write performance.
- **Background jobs** — implement when campaign sends or report generation start causing API timeouts.